

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: Karthik Chelluri

Roll No.: A24126552077

Date: 25-08-2026

1. TITLE

Implementation of JavaScript Arrays, Object-Oriented Programming and Functions

2. AIM

To implement and understand basic JavaScript concepts including arrays and array methods, Object-Oriented Programming using classes, private fields and methods, and functions with parameters.

3. OBJECTIVE

The core objectives of this laboratory experiment are:

- To create and manipulate arrays in JavaScript.
- To access array elements using index numbers.
- To find the length of an array using the length property.
- To update array elements dynamically.
- To use push(), pop(), shift(), and unshift() methods.
- To iterate through an array using a for...of loop.
- To understand classes and objects in JavaScript.
- To implement encapsulation using private fields (#balance).
- To create constructors and object-oriented methods.
- To create and use reusable functions with parameters.
- To perform complex operations on an array using modular functions.

4. THEORY

JavaScript provides several core features for handling data, encapsulating operations, and structuring modular programs. This experiment focuses on three fundamental concepts: Arrays, Object-Oriented Programming (OOP), and Functions.

A. Arrays: An array is a collection of multiple values stored in a single variable. Array elements are ordered and accessed using zero-based index numbers. JavaScript arrays are dynamic and come with powerful built-in methods for data manipulation:

- `push()` - Adds one or more elements to the end of an array and returns the new length.
- `pop()` - Removes the last element from an array and returns that element.
- `shift()` - Removes the first element from an array and returns that element.
- `unshift()` - Adds one or more elements to the beginning of an array and returns the new length.

B. Object-Oriented Programming (OOP): OOP is a programming paradigm centered on objects rather than logic and functions. A class serves as a blueprint for creating structured objects. Encapsulation is enforced using private fields, declared with the `#` symbol. These private fields cannot be accessed directly from outside the class boundary, protecting internal states and allowing mutations only via dedicated getter and setter methods.

C. Functions: A function is a self-contained, reusable block of code designed to perform a specific task. By passing parameters, functions can process varied arguments dynamically. Functions make code modular, easily maintainable, and significantly reduce redundancy.

5. ALGORITHM / PROCEDURE

The development and validation procedure is divided into three parts:

1. Create a JavaScript file named `array.js`.
2. Create an array containing three initial colors ('Red', 'Green', 'Blue').
3. Display the original list and access individual elements using indices `colors[0]` and `colors[1]`.
4. Find and print the total array length using `colors.length`.
5. Update the element at index 1 to 'Yellow'.
6. Append 'Purple' to the end of the array using `push()` and remove it using `pop()`.
7. Remove the first element using `shift()` and prepend 'Pink' using `unshift()`.
8. Iterate over the array using a `for...of` loop and display each color.
9. Create an Object-Oriented program in `oops.js`.
10. Define a `BankAccount` class with a private field `#balance` initialized to 0.
11. Create a constructor accepting 'owner' and getter/setter methods: `getBalance()` and `deposit(amount)`.
12. Instantiate a `BankAccount` object with owner 'Alex', call `deposit(500)`, and display the final balance.

13. 13. Create a modular program in functions.js to model a shopping cart.
14. 14. Define shoppingCart array containing 'Laptop', 'Headphones', and 'Mouse Pad'.
15. 15. Create functions: viewCart() to display items, addItem() to push new items, and removeLastItem() to pop elements.
16. 16. Execute and test the shopping cart operations in sequence to verify arrays under functional contexts.

6. PROGRAM

File Name: array.js

```
// 1. Creating an array with initial values
const colors = ["Red", "Green", "Blue"];
console.log("Original List:", colors);

// 2. Accessing items using index numbers
console.log("First color (index 0):", colors[0]);
console.log("Second color (index 1):", colors[1]);

// 3. Finding the total number of items
console.log("Total colors in list:", colors.length);

// 4. Updating an item
colors[1] = "Yellow";
console.log("After updating index 1:", colors);

// 5. Adding a new item to the END
colors.push("Purple");
console.log("After adding to the end:", colors);

// 6. Removing the last item
const removedColor = colors.pop();
console.log("Removed color:", removedColor);
console.log("Array after removing last item:", colors);

// Removing the first element
const remove = colors.shift();
console.log("Remove:", remove);

// Adding an element at the beginning
colors.unshift("Pink");
console.log("Adding at start:", colors);

// 7. Looping through the array
console.log("\n--- Printing all colors one by one ---");
for (const color of colors) {
    console.log(`Color: ${color}`);
}
```

File Name: oops.js

```
class BankAccount {
  // Private field
  #balance = 0;

  constructor(owner) {
    this.owner = owner;
  }

  // Getter-like method
  getBalance() {
    return this.#balance;
  }

  // Setter-like operation
  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited ${amount}.`);
    } else {
      console.log("Invalid deposit amount.");
    }
  }
}

// Creating an object
const myAccount = new BankAccount("Alex");

// Depositing money
myAccount.deposit(500);

// Displaying balance
console.log(myAccount.getBalance());
```

File Name: functions.js

```
const shoppingCart = [
  "Laptop",
  "Headphones",
  "Mouse Pad"
];

// Function to print all items
function viewCart(cartArray) {
  console.log(`\nYour Cart (${cartArray.length} items)`);
  if (cartArray.length === 0) {
    console.log("Your cart is empty!");
  }
}
```

```

    return;
  }
  for (let i = 0; i < cartArray.length; i++) {
    console.log(`${i + 1}. ${cartArray[i]}`);
  }
}

// Function to add a new item
function addItem(cartArray, item) {
  cartArray.push(item);
  console.log(`Added to cart: ${item}`);
}

// Function to remove the last item
function removeLastItem(cartArray) {
  if (cartArray.length > 0) {
    const removed = cartArray.pop();
    console.log(`Removed from cart: ${removed}`);
  } else {
    console.log("Nothing to remove. Cart is already empty!");
  }
}

// Running the program
viewCart(shoppingCart);
addItem(shoppingCart, "Keyboard");
viewCart(shoppingCart);
removeLastItem(shoppingCart);
viewCart(shoppingCart);

```

7. CODE EXPLANATION

Keyword / Statement	Functional Explanation
<code>colors = []</code>	Creates an array containing initial color string values.
<code>colors[0]</code>	Accesses the element in the array at position 0 ('Red').
<code>colors.length</code>	Finds the total count of elements inside the active array container.
<code>colors[1] = 'Yellow'</code>	Replaces the original value at index 1 with 'Yellow' dynamically.

<code>push()</code>	Adds a given element as the last item inside the array.
<code>pop()</code>	Removes the last item in the array and returns its value.
<code>shift()</code>	Removes the first item in the array and shifts subsequent items left.
<code>unshift()</code>	Inserts a new element at the very beginning (index 0) of the array.
<code>for...of</code>	A clean loop that iterates directly through each item inside the array container.
<code>class BankAccount</code>	Defines an object blueprint containing constructor states and methods.
<code>#balance = 0</code>	Declares an encapsulated private field that is inaccessible outside the class scope.
<code>constructor(owner)</code>	Initializes the object instances, mapping the owner value on load.
<code>this.owner</code>	Stores and references the owner name attribute inside the object state.
<code>getBalance()</code>	Getter method that acts as a secure read channel for private <code>#balance</code> .
<code>deposit(amount)</code>	Setter method incorporating validation to securely update internal balance state.
<code>viewCart()</code>	A function displaying all index-mapped values inside the shopping cart array.
<code>addItem()</code>	Pushes a new item into the shopping cart array dynamically.
<code>removeLastItem()</code>	Pops the last item out of the cart array, validating for empty conditions.

8. TEST CASES AND EXECUTION DATA

A. Array Program Validation (array.js)

Test ID	Operation	Expected Result	Actual Result	Status
TC-01	Create colors array	Red, Green, Blue displayed	Red, Green, Blue displayed	Pass
TC-02	Access index 1	Green displayed	Green displayed	Pass
TC-03	Check length of list	Length 3 displayed	Length 3 displayed	Pass
TC-04	Use push() method	Purple added to end	Purple added to end	Pass

B. Object-Oriented Program Validation (oops.js)

Test ID	Operation	Expected Result	Actual Result	Status
TC-01	Create BankAccount object	Object created successfully	Object created successfully	Pass
TC-02	Deposit 500 into account	500 deposited successfully	500 deposited successfully	Pass
TC-03	Get balance secure check	Balance should display 500	Balance displays 500	Pass
TC-04	Deposit negative amount	Invalid deposit amount alert	Invalid message displayed	Pass

C. Shopping Cart Functions Validation (functions.js)

Test ID	Operation	Expected Result	Actual Result	Status
TC-01	View cart on init	Three items displayed clearly	Three items displayed clearly	Pass

TC-02	Add item 'Keyboard'	Keyboard added successfully	Keyboard added successfully	Pass
TC-03	View updated cart	Four items displayed with Keyboard	Four items displayed with Keyboard	Pass
TC-04	Remove last item	Keyboard removed successfully	Keyboard removed successfully	Pass

9. OUTPUT SCREENSHOTS

9.1 Array Program Output (array.js)

Below is the console terminal screenshot displaying the complete array lifecycle, index mapping, length check, item updates, pop/push operations, and item iteration through a custom loop:

```
powershell - Karthik Chelluri (A24126552077)

PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3> node array.js
Original List: [ 'Red', 'Green', 'Blue' ]
First color (index 0): Red
Second color (index 1): Green
Total colors in list: 3
After updating index 1: [ 'Red', 'Yellow', 'Blue' ]
After adding to the end (.push): [ 'Red', 'Yellow', 'Blue', 'Purple' ]
Removed color: Purple
Array after removing last item (.pop): [ 'Red', 'Yellow', 'Blue' ]
Remove: Red
adding at start [ 'Pink', 'Yellow', 'Blue' ]

--- Printing all colors one by one ---
Color: Pink
Color: Yellow
Color: Blue
PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3>
```

Figure 9.1: Console output showing array.js execution details.

9.2 OOP encapsulation check (oops.js)

Below is the terminal execution check of oops.js validating the instantiation of a class and successful deposits to the private field through secure methods:

```
powershell - Karthik Chelluri (A24126552077)

PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3> node oops.js
Deposited 500.
500
PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3>
```

Figure 9.2: Secure object instantiation and balance validation in oops.js.

9.3 Shopping Cart Functions (functions.js)

Below is the terminal output confirming modular operations inside functions.js. The console logs clear execution outputs for viewing the cart, adding an item, and safely removing the last item:

```
powershell - Karthik Chelluri (A24126552077)

PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3> node functions.js

Your Cart (3 items)
1. Laptop
2. Headphones
3. Mouse Pad
Added to cart: Keyboard

Your Cart (4 items)
1. Laptop
2. Headphones
3. Mouse Pad
4. Keyboard
Removed from cart: Keyboard

Your Cart (3 items)
1. Laptop
2. Headphones
3. Mouse Pad
PS C:\Users\Karthik_A24126552077\FSWD_Lab\week-3>
```

Figure 9.3: Function execution and list manipulation logs in functions.js.

10. RESULT

Thus, the JavaScript programs were successfully implemented and executed in accordance with standard classroom protocols. The array program demonstrated creation, indexing, updating, and methods like push(), pop(), shift(), and unshift() successfully. The Object-Oriented class (oops.js) demonstrated secure constructor mapping, method queries, and private #balance encapsulation validation. Finally, the shopping cart program (functions.js) confirmed the functional logic of modular parameter passing. All components compiled and executed cleanly, matching the required academic outcomes.